

Cours complet : les réseaux de neurones

Mathématiques et informatique — projet chessXvi

Ce cours part de zéro. Il explique à la fois les mathématiques (ce qui se passe vraiment) et l'informatique (comment on le code, en lien avec `Layer.java` et `Network.java`). L'objectif est que tu comprennes tout, sans trou.

Tableau des chapitres

#	Chapitre	État
0	Intuition : c'est quoi une IA ?	✓
1	Le neurone formel	✓ codé
2	Les fonctions d'activation	✓ codé
3	Rappels de maths : vecteurs & matrices	✓
4	La couche de neurones	✓ codé
5	Le réseau complet (propagation avant)	✓ codé
6	Faire « apprendre » : la fonction de coût	à faire
7	Rappels de maths : dérivées & gradient	à faire
8	La descente de gradient	à faire
9	La rétropropagation	à faire
10	L'entraînement en pratique	à faire
11	Application aux échecs	à faire

Où on en est : voir la dernière section, « Où on se situe ».

1 Intuition : c'est quoi une IA ?

Dans ce cours, « intelligence artificielle » désigne un objet mathématique précis : un **réseau de neurones**. Ce n'est pas magique : c'est une **fonction**. Une fonction prend des nombres en **entrée** et renvoie des nombres en **sortie** :

$$\{2, 3\} \longrightarrow \boxed{\text{FONCTION}} \longrightarrow \{6,5\}$$

La particularité d'un réseau de neurones est de contenir **beaucoup de réglages** (les « poids »). « Entraîner une IA », c'est simplement **trouver les bons réglages** pour que la sortie soit celle qu'on veut. Tout le cours répond à deux questions :

1. Comment la fonction calcule sa sortie ? (chap. 1 à 5 : la *propagation avant*)
2. Comment trouver les bons réglages ? (chap. 6 à 10 : l'*apprentissage*)

2 Le neurone formel

2.1 L'idée

Un **neurone** est la plus petite brique. Il reçoit plusieurs nombres en entrée et en produit **un seul** en sortie. Il fait trois choses :

1. il **pondère** chaque entrée (chaque entrée a un *poids*) ;
2. il **additionne** le tout, plus un nombre fixe appelé *biais* ;
3. il passe le résultat dans une *fonction d'activation* (chap. 2).

2.2 Les mathématiques

Notons les entrées $x_1, \dots, x_n$, les poids $w_1, \dots, w_n$ et le biais b . Le neurone calcule d'abord la **pré-activation** z :

$$z = b + w_1x_1 + w_2x_2 + \dots + w_nx_n = b + \sum_{i=1}^n w_i x_i$$

- w_i grand $\Rightarrow$ l'entrée x_i compte beaucoup ;
- w_i négatif $\Rightarrow$ l'entrée joue « contre » ;
- b (biais) décale le résultat (comme l'ordonnée à l'origine d'une droite).

Puis on applique l'activation f pour obtenir la sortie a :

$$a = f(z)$$

2.3 Le lien avec le code

Dans `Layer.java`, pour **un** neurone `n` :

```
double z = biases[n];           // z = b
for (int i = 0; i < input.length; i++) {
    z += weights[n][i] * input[i]; // z += w_i * x_i
}
output[n] = relu(z);           // a = f(z)
```

À retenir. Un neurone = biais + somme(poids $\times$ entrées), puis activation.

3 Les fonctions d'activation

3.1 Pourquoi en a-t-on besoin ?

Sans activation, un neurone n'est qu'une **somme pondérée**, donc une opération **linéaire**. Empiler des couches linéaires reste linéaire : on ne pourrait représenter que des droites, même avec mille couches. La fonction d'activation introduit de la **non-linéarité**, ce qui permet d'apprendre des formes complexes (courbes, frontières, motifs).

3.2 ReLU (celle utilisée dans le projet)

$$\text{ReLU}(z) = \max(0, z)$$

Si z est négatif on renvoie 0, sinon on garde z .

```
double relu(double z) {
    return Math.max(0, z);
}
```

C'est la plus utilisée aujourd'hui : simple, rapide, efficace.

3.3 La sigmoïde (à connaître)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Elle « écrase » tout nombre entre 0 et 1. Pratique pour produire une **probabilité** (ex. « probabilité que ce coup soit bon = 0,87 »).

À retenir. L'activation rend le réseau capable d'apprendre autre chose que des droites. ReLU = $\max(0, z)$.

4 Rappels de maths : vecteurs et matrices

C'est le **langage** des réseaux.

4.1 Un vecteur

Une **liste ordonnée de nombres**. En code : un `double[]`.

$$x = \begin{pmatrix} 2 \\ 3 \end{pmatrix} \iff \text{double[] } x = \{2, 3\};$$

4.2 Une matrice

Un **tableau de nombres à deux dimensions** (lignes $\times$ colonnes). En code : un `double[][]`.

$$W = \begin{pmatrix} 0,5 & -1 \\ 1 & 1 \\ -1 & 0,5 \end{pmatrix}$$

Cette matrice a **3 lignes** et **2 colonnes**. Dans un réseau : **1 ligne = 1 neurone**, et le **nombre de colonnes = le nombre d'entrées**. Elle décrit donc 3 neurones attendant chacun 2 entrées : c'est exactement `layer1` dans `Network.java`.

4.3 Le produit matrice $\times$ vecteur

C'est l'**opération centrale**. Chaque ligne de W est combinée avec x par une somme pondérée :

$$Wx = \begin{pmatrix} 0,5 & -1 \\ 1 & 1 \\ -1 & 0,5 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 0,5 \cdot 2 + (-1) \cdot 3 \\ 1 \cdot 2 + 1 \cdot 3 \\ -1 \cdot 2 + 0,5 \cdot 3 \end{pmatrix} = \begin{pmatrix} -1 \\ 5 \\ -0,5 \end{pmatrix}$$

Chaque ligne du résultat est exactement la somme pondérée d'un neurone : le produit matrice $\times$ vecteur **calcule tous les neurones d'une couche d'un coup**. En ajoutant les biais b puis l'activation, on obtient toute la couche :

$$a = f(Wx + b)$$

Retiens cette formule : c'est tout un réseau, répété couche après couche.

5 La couche de neurones

Une **couche** (Layer) regroupe des neurones qui reçoivent les **mêmes entrées** et produisent **chacun une sortie** : c'est la formule $a = f(Wx + b)$.

Maths	Code (Layer.java)	Signification
W	<code>weights (double[] [])</code>	1 ligne par neurone
b	<code>biases (double[])</code>	1 biais par neurone
x	<code>input (double[])</code>	le vecteur d'entrée
$Wx + b$	la variable <code>z</code>	la pré-activation
f	<code>relu(...)</code>	l'activation
a	<code>output (double[])</code>	le vecteur de sortie

```
public double[] forward(double[] input) {
    int neurons = weights.length;           // nb de lignes de W = nb de neurones
    double[] output = new double[neurons];
    for (int n = 0; n < neurons; n++) {      // pour chaque neurone (ligne de W)
        double z = biases[n];                // z = b[n]
        for (int i = 0; i < input.length; i++) {
            z += weights[n][i] * input[i];    // z += W[n][i] * x[i]
        }
        output[n] = relu(z);                 // a[n] = f(z)
    }
    return output;
}
```

La double boucle fait *à la main* le produit matrice $\times$ vecteur : la boucle `n` parcourt les lignes (les neurones), la boucle `i` parcourt les colonnes (les entrées).

5.1 Exemple chiffré (la couche 1 du projet)

Entrée $x = \{2, 3\}$, matrice W ci-dessus, biais $b = \{1, 0, 2\}$:

Neurone	$z = b + \sum w_i x_i$	$a = \text{ReLU}(z)$
0	$1 + 0,5 \cdot 2 - 1 \cdot 3 = -1$	0
1	$0 + 1 \cdot 2 + 1 \cdot 3 = 5$	5
2	$2 - 1 \cdot 2 + 0,5 \cdot 3 = 1,5$	1,5

Sortie de la couche : $\{0, 5, 1,5\}$.

6 Le réseau complet (propagation avant)

Un **réseau** (Network) est une **suite de couches** : la sortie d'une couche devient l'entrée de la suivante. C'est la **propagation avant** (*forward pass*).

$$x \xrightarrow{\text{couche 1}} a^{(1)} \xrightarrow{\text{couche 2}} a^{(2)} \longrightarrow \dots \longrightarrow \text{sortie}$$

```
public double[] forward(double[] input) {
    double[] current = input;           // on part de l'entree
    for (int k = 0; k < layers.length; k++) { // pour chaque couche...
```

```

    current = layers[k].forward(current);    // ...la sortie devient la nouvelle entree
}
return current;
}

```

La variable `current` est le **relais** qui circule de couche en couche.

6.1 Exemple chiffré complet (le réseau $2 \rightarrow 3 \rightarrow 1$)

- **Couche 1** : entrée $\{2, 3\} \rightarrow \{0, 5, 1, 5\}$ (calcul précédent).
- **Couche 2** : 1 neurone, poids $\{1, 1, 1\}$, biais 0 :

$$z = 0 + 1 \cdot 0 + 1 \cdot 5 + 1 \cdot 1,5 = 6,5 \quad \Rightarrow \quad \text{ReLU}(6,5) = 6,5$$

- **Sortie finale** : $\{6,5\}$.

À ce stade, le réseau sait **calculer** une sortie : c'est ce que fait ton code aujourd'hui. **Mais les poids sont posés à la main** : le réseau ne sait pas encore *apprendre*. C'est l'objet des chapitres suivants.

7 Faire « apprendre » : la fonction de coût

7.1 Le problème

Apprendre, c'est laisser la machine **trouver les poids toute seule**, à partir d'**exemples**. Un exemple = une entrée + la **bonne réponse attendue** (la « cible », notée y).

7.2 Mesurer l'erreur

Le réseau produit une prédiction $\hat{y}$. On la compare à la cible y avec une **fonction de coût** (ou *perte*). La plus simple est l'**erreur quadratique** :

$$C = (\hat{y} - y)^2$$

- si $\hat{y} = y$: coût 0 (parfait) ;
- plus on est loin, plus le coût est grand (le carré pénalise fort les gros écarts).

Idée-clé. Apprendre = rendre C le plus petit possible en ajustant les poids et les biais. C'est un problème de **minimisation**. L'outil pour minimiser une fonction, c'est la **dérivée** (chap. 7).

8 Rappels de maths : dérivées et gradient

8.1 La dérivée : la « pente »

La **dérivée** d'une fonction en un point est sa **pente** : elle dit dans quel sens et à quelle vitesse la fonction monte ou descend.

- dérivée **positive** : la fonction monte ;
- dérivée **négative** : la fonction descend ;
- dérivée **nulle** : on est sur un plat (sommet ou creux).

Notation : $\frac{dC}{dw}$ « de combien varie le coût C si je bouge un peu le poids w ».

8.2 Le gradient : la pente en plusieurs dimensions

Quand le coût dépend de beaucoup de poids, on prend la dérivée par rapport à **chacun** (les *dérivées partielles*, notées ∂). Le **gradient** les rassemble :

$$\nabla C = \left(\frac{\partial C}{\partial w_1}, \frac{\partial C}{\partial w_2}, \dots \right)$$

Propriété fondamentale : le gradient **pointe dans la direction où le coût augmente le plus vite**. Pour **diminuer** le coût, on va donc dans le sens **opposé** au gradient (chap. 8).

8.3 Une dérivée dont on aura besoin

$$C = (\hat{y} - y)^2 \implies \frac{dC}{d\hat{y}} = 2(\hat{y} - y)$$

(la dérivée de « quelque chose au carré » = $2 \times$ ce quelque chose $\times$ sa propre dérivée.)

9 La descente de gradient

C'est l'**algorithme d'apprentissage**. Image : tu es dans la montagne, dans le brouillard, et tu veux rejoindre la vallée (le coût minimal). Tu ne vois pas loin, mais tu sens la **pente sous tes pieds** (le gradient). Stratégie : faire un **petit pas vers le bas**, et recommencer.

Pour chaque poids w :

$$w \leftarrow w - \eta \frac{\partial C}{\partial w}$$

- $\frac{\partial C}{\partial w}$: la pente (le gradient) pour ce poids ;
- le signe **moins** : on descend (sens opposé au gradient) ;
- η (« éta »), le **taux d'apprentissage** : la taille du pas. Trop petit $\Rightarrow$ très lent ; trop grand $\Rightarrow$ on saute par-dessus la vallée, ça diverge.

On répète des milliers de fois : à chaque tour le coût baisse un peu. Reste à calculer toutes ces dérivées partielles dans un réseau à plusieurs couches (chap. 9).

10 La rétropropagation (*backpropagation*)

10.1 Le défi

Un poids de la **première** couche influence la sortie en traversant **toutes** les couches suivantes. Comment connaître sa part de responsabilité dans l'erreur finale ?

10.2 La règle de la chaîne

Si une grandeur en influence une autre qui en influence une troisième, on **multiplie les dérivées** le long du chemin :

$$\frac{dC}{dw} = \frac{dC}{da} \cdot \frac{da}{dz} \cdot \frac{dz}{dw}$$

10.3 L'idée de l'algorithme

1. **Propagation avant** : calculer la sortie (déjà fait par ton code) en mémorisant les valeurs intermédiaires (z et a de chaque couche).
2. **Calcul de l'erreur finale** C (chap. 6).

3. **Propagation arrière** : partir de la sortie et **remonter** couche par couche, en multipliant les dérivées, pour obtenir $\partial C / \partial w$ de **chaque** poids.
4. **Mise à jour** de tous les poids par descente de gradient (chap. 8).

C'est le « back » de *backpropagation* : l'information de l'erreur circule **à l'envers**, de la sortie vers l'entrée.

Dérivée utile : pour ReLU, $\text{ReLU}'(z) = 1$ si $z > 0$, et 0 sinon. Très simple à coder, d'où sa popularité.

11 L'entraînement en pratique

- **Données d'entraînement** : l'ensemble des exemples (entrée + cible).
- **Itération** : un passage avant + arrière + mise à jour.
- **Batch** (lot) : le paquet d'exemples traités avant chaque mise à jour.
- **Epoch** (époque) : un passage complet sur toutes les données.
- **Learning rate** η : la taille du pas, le réglage le plus important.
- **Surapprentissage** (*overfitting*) : le réseau apprend « par cœur » au lieu de **généraliser**. On le détecte avec des données de **test** jamais vues.

```
repete pour chaque epoch :
  pour chaque exemple (x, y) :
    y_chapeau = reseau.forward(x)    // propagation avant
    C = cout(y_chapeau, y)           // erreur
    gradients = backprop(C)          // propagation arriere
    mettre a jour les poids           // w <- w - eta * gradient
```

12 Application aux échecs

But final : un réseau qui **évalue une position** (ou choisit un coup). À régler dans l'ordre :

1. **Encodage de l'entrée** : transformer l'échiquier (le `String[] [] squares` de `Board.java`) en un **vecteur de nombres**. Ex. : case vide = 0, pion blanc = +1, dame noire = -9, etc.
2. **Définir la sortie** : un seul nombre (évaluation) ? une probabilité par coup ?
3. **Obtenir des données** : des positions avec leur « bonne » évaluation.
4. **Entraîner** avec les chapitres 6 à 10.
5. **Brancher** le réseau entraîné sur le jeu.

Où on se situe dans le cours

```
[0]-[1]-[2]-[3]-[4]-[5]-| TU ES ICI |-[6]-[7]-[8]-[9]-[10]-[11]
OK OK OK OK OK OK          a faire .....
|----- PROPAGATION AVANT -----| |----- APPRENTISSAGE -----|
  (déjà code : Layer.java + Network.java)   (pas encore commence)
```

Acquis (chap. 0 à 5), codé dans `Layer.java` / `Network.java` :

- le neurone : `biais + somme(poids × entrées)` ;
- la fonction d'activation ReLU ;

- la couche, vue comme un produit matrice $\times$ vecteur ;
- le réseau et la propagation avant.

Concrètement, on s’est arrêté à la fin du chapitre 5. Ton réseau sait **calculer** une sortie, mais les poids sont **fixés à la main** : il ne sait pas encore **apprendre**.

Prochaine étape (chapitre 6) : introduire la **fonction de coût** pour **mesurer l’erreur**. C’est le premier pas vers l’apprentissage ; viendront ensuite les dérivées/gradient (7), la descente de gradient (8) et la rétropropagation (9), qui permettront enfin au réseau d’ajuster ses poids tout seul.